

Projekt, 1. část: UPA

Matúš Biely xbiely00@stud.fit.vutbr.cz
Ondřej Misař xmisar01@stud.fit.vutbr.cz
Samuel Kuchta xkucht11@stud.fit.vutbr.cz

Október 2023

Obsah

1	Stĺpcová wide-column databáza	1
1.1	Vysvetlenie zvolenia databázy	1
1.2	Definícia úložiska	1
1.3	Import dát	2
1.4	Dotaz nad databázou	3
2	Databáza časových radov	4
2.1	Vysvetlenie zvolenia databázovej technológie	4
2.2	Definícia úložiska	4
2.3	Import dat	5
2.4	Dotaz nad databázou	5
3	Grafová databáze	6
3.1	Vysvětlení zvolení databáze	6
3.2	Definice úložiště	6
3.3	Import dat	7
3.4	Dotaz nad databazí	8
4	Dokumentová databáza	9
4.1	Vysvetlenie zvolenia databázy	9
4.2	Definícia úložiska	9
4.3	Import dát	10
4.4	Dotaz nad databázou	11

1 Stĺpcová wide-column databáza

Dataset: Intenzity cyklistů z cyklo detektorů - aktuální

URL: <https://data.gov.cz/datov%C3%A1-sada?iri=https%3A%2F%2Fdata.gov.cz%2Fzdroj%2Fdatov%C3%A9-sady%2F44992785%2F0d6f1518d017cc26385f1f9c4bc36342>

Distribúcia dátovej sady: Je dostupná ako CSV, JSON, KML aj Shapefile. Použijeme CSV.

Zvolený produkt: Apache Cassandra

Vychádzali sme z oficiálnej dokumentácie: <https://cassandra.apache.org/doc/latest/>

1.1 Vysvetlenie zvolenia databázy

Štruktúra dátovej sady je taká, že obsahuje informácie z cyklo detektorov umiestnených na rôznych miestach v meste Brno. Každý záznam dát zahŕňa podrobné informácie o daných cyklo detektoroch, ako identifikátor, poloha, sledovaný časový interval a smery cesty, ktoré sleduje. Pre oba sledované smery cesty sú obsiahnuté informácie o počte cyklistov, chodcov a cyklistov a chodcov spolu.

NoSQL databáza Cassandra je vhodná na použitie za predpokladu, že dátová sada je často aktualizovaná a má spracovávať veľké množstvo dát. Periodicita aktualizácie tejto dátovej sady je raz za hodinu, na základe čoho vieme usúdiť, že pre túto dátovú sadu je vhodné použiť práve NoSQL databázu Cassandra. Ďalšou výhodou použitia tejto databázy je fakt, že databáza Cassandra obsahuje tabuľky, ktoré sú indexované pomocou tzv. „partition“ a „clustering“ kľúčov. Partition kľúč nám delí tabuľku na menšie tabuľky za použitia hash funkcie, a clustering kľúč nám určuje, ako sa vyhľadávajú dáta v danej partícii pomocou B+ stromu. Pri periodicite dát v hodinových intervaloch vieme povedať, že jednotlivé cyklo detektory budú predstavovať naše „partition“ kľúče a jednotlivé intervaly nahraných dát budú predstavovať „clustering“ kľúče.

Jedná sa o namerané dáta, preto nepredpokladáme, že by dochádzalo k častej úprave importovaných dát. Dátová sada neobsahuje chýbajúce záznamy, nakoľko v prípade chýbajúcich dát dátová sada obsahuje informácie pre každý cyklo detektor zo staršieho dátumu ako náhrada chýbajúcich dát. V prípade, že sa nebude jednať o reprezentatívne dáta bude ich možné zmeniť, ak sa bude jednať o "náhradné" dáta.

Databáza Cassandra nám umožní rýchle spracovanie často aktualizovaných dát a rýchle vyhľadávanie dát pre konkrétny cyklo detektor v hľadanom časovom intervale.

Cassandra podporuje zabúdanie dát nastavením platnosti záznamu (TTL - Time to live). Bolo by potrebné navrhnuť vhodné časové obdobie, počas ktorého si chceme pamätať namerané údaje z cyklo detektorov. Toto obdobie by mohlo predstavovať posledný deň, týždeň, mesiac, či rok.

Optimalizáciu v tabuľkách je možné zabezpečiť úpravou partition a clustering kľúčov. Túto optimalizáciu je však nutné zabezpečiť už pri vytváraní tabuliek, nakoľko databáza Cassandra nepodporuje dynamické zmeny tabuliek. Pre tento typ databázy je nutné zabezpečiť optimalizáciu už pri modelovaní, preto je dôležité si dopredu premyslieť spôsob uloženia dát a na základe analýzy navrhnuť vhodný model.

1.2 Definícia úložiska

Dáta sa v Cassandre ukladajú podobne ako v relačnej databáze do tzv. „keyspace“, čo je obdobie „database“ v relačných databázových systémoch.

Pri vytváraní „keyspace“ je potrebné špecifikovať dva základné parametre, aby bola zaručená správna a požadovaná replikácia na jednotlivých uzloch. Tieto replikačné stratégie delíme na:

- Jednoduchá stratégia a
- stratégia sieťovej topológie

Jednoduchá stratégia je vhodná na použitie, ak neplánujeme nasadenie clusteru na viac než jedno dátové centrum. Replikačný faktor, čo je nastavenie toho, koľko replík jedného riadku sú uložené na jednom uzle, je pri použití tejto stratégie aplikovaný na celý cluster.

Stratégia sieťovej topológie je vhodná na použitie, keď potrebujeme nasadiť náš cluster na viacero dátových centier. Každé dátové centrum bude mať samostatný replikačný faktor.

Pre naše použitie budeme pracovať s jednoduchou replikačnou stratégiou, nakoľko pri jednej relatívne menšej dátovej sade zatiaľ nie je potrebné navrhovať topológiu replikácie pre viacero dátových centier. Replikačný faktor zvolíme primerane, aby nedochádzalo ku zbytočnému zahlcovaniu databázy, ale aj aby sme zabezpečili vhodnú redundanciu.

```
CREATE KEYSPACE IF NOT EXISTS ks_cyklo
WITH replication = {'class':'SimpleStrategy', 'replication_factor' : 3};
USE ks_cyklo;
```

Ako už bolo vyššie spomínané, pre túto dátovú sadu vytvoríme tabuľku cyklodetektorov s partition kľúčom na identifikátor daného cyklodetektora a identifikátor lokácie (detektor by sa potenciálne mohol aj presunúť, čím by sme chceli dostať odlišný index pre vyhľadávanie). Navyše je potrebné pridať do partition kľúču tretiu položku, vďaka ktorej si nebudeme prepisovať staré dáta novými. Preto pridáme do partition kľúču aj položku EndOfInterval, ktorá nám zaručí, že dáta ktoré sa zmenili od poslednej aktualizácie neprepíšu už existujúce dáta. Clustering kľúč bude stĺpec „datum“, ktorý predstavuje dátum a čas poslednej aktualizácie dátovej sady. Tabuľka bude vrámci jednotlivých partícií radená v zostupnom poradí na základe stĺpcu „datum“.

```
CREATE TABLE IF NOT EXISTS t_cyklo (
    ObjectId int ,
    UnitName text ,
    UnitId text ,
    LocationId int ,
    Latitude double ,
    Longitude double ,
    EndOfInterval timestamp ,
    FirstDirection_Name text ,
    FirstDirection_Cyclists int ,
    FirstDirection_Pedestrians int ,
    FirstDirection_Total int ,
    SecondDirection_Name text ,
    SecondDirection_Cyclists int ,
    SecondDirection_Pedestrians int ,
    SecondDirection_Total int ,
    datum timestamp ,
    PRIMARY KEY ((ObjectId , LocationId , EndOfInterval) , datum)
) WITH CLUSTERING ORDER BY (datum DESC);
```

1.3 Import dát

Import dát do databázy Cassandra sa najčastejšie používa za použitia CSV súborov, ktoré sú dostupné pre zvolenú dátovú sadu. Predpokladajme teda, že CSV súbor máme uložený v súbore `data.csv`, ktorý sme tam uložili my, prípadne je tento súbor periodicky prepisovaný pri aktualizácii dátovej sady. Taktiež predpokladajme, že sme už použili príkaz `USE ks_cyklo` pre zvolenie keyspace-u, v ktorom máme vytvorenú tabuľku `t_cyklo`. Následne vieme po pripojení sa do Cassandra clusteru pomocou `cqlsh` spustiť nahranie dát z CSV súboru za pomoci príkazu:

```
COPY t_cyklo (
    ObjectId ,
    UnitName ,
    UnitId ,
    LocationId ,
    Latitude ,
    Longitude ,
    EndOfInterval ,
    FirstDirection_Name ,
    FirstDirection_Cyclists ,
    FirstDirection_Pedestrians ,
    FirstDirection_Total ,
    SecondDirection_Name ,
    SecondDirection_Cyclists ,
    SecondDirection_Pedestrians ,
    SecondDirection_Total ,
    datum
) FROM 'data.csv' WITH HEADER = false ;
```

Nakoľko sme navrhli „partition“ kľúč zložený z troch častí, pri kopírovaní, či inserte dát do tabuľky s rovnakým už existujúcim „partition“ kľúčom sú dáta v tabuľke aktualizované. V prípade, že neexistujú sú tieto dáta vložené.

Príklad insertu:

```
INSERT INTO t_cyklo (ObjectId, LocationId, datum)
VALUES (1, 1, '2023-10-23 08:00:00+00')
```

Príklad updatu:

```
UPDATE t_cyklo
SET Longitude = 49.1836985561504, Latitude = 16.6024231910706
WHERE ObjectId = 1 AND LocationId = 0 AND datum = '2023-10-23 08:00:00+00';
```

1.4 Dotaz nad databázou

Ako príklad pre dotaz nad databázou zvolíme dotaz, ktorý nám vyfiltruje dáta pre konkrétny cyklo-detektor umiestnený na konkrétnom mieste, a vyfiltrujeme si dáta za posledný mesiac. Dáta budú zoradené vzostupne podľa stĺpcu `datum` nakoľko sme si zvolili takéto poradie pri vytváraní tabuľky.

```
SELECT *
FROM t_cyklo
WHERE   ObjectId = 1
        AND LocationId = 1
        AND datum >= dateOf(now()) - 30
        AND datum <= dateOf(now());
```

Ako bolo už vyššie napísané vyhľadávanie v databáze Cassandra prebieha pomocou hash vyhľadávania na základe „partition“ kľúča a pomocou B+ stromu v danej partícii na základe „clustering“ kľúča. Preto sa v tomto dotaze najskôr vyhodnotí `ObjectId` a `LocationId` a vyhľadajú sa dáta pomocou hash funkcie. Po tom prebehne v danej partícii tabuľky vyhľadávanie na základe `datum` pomocou B+ stromu.

2 Databáza časových radov

Dataset: Kvalita ovzduší

URL: <https://data.gov.cz/datov%C3%A1-sada?iri=https%3A%2F%2Fdata.gov.cz%2Fzdroj%2Fdatov%C3%A9-sady%2F44992785%2F4907e5c199620d99a44c1ec388703cd5>

Distribúcia dátovej sady je dostupná ako CSV, GeoJSON, KML a Shapefile.

Zvolený produkt: InfluxDB

2.1 Vysvetlenie zvolenia databázovej technológie

Tu sú niektoré dôvody, prečo je InfluxDB vhodnou voľbou pre dátovú sadu o kvalite ovzdušia v staniciach na území mesta Brna:

InfluxDB vyniká špecializovaným zameraním na prácu s časovými radmi, vďaka čomu je optimálnou voľbou na ukladanie a dopytovanie údajov, ktoré sa časom menia (napr. zmeny zloženia vzduchu v časopriestore).

Jednou z výrazných výhod InfluxDB je jeho jednoduchá štruktúra a jednoduché použitie. Databázový systém ponúka jednoduchý a intuitívny spôsob ukladania a analýzy časových radov, čím uľahčuje prácu s údajmi a skracaje čas potrebný na získanie hodnotných výsledkov. To sa hodí napr. pre kontrolovanie, či nenastala nejaká porucha v priemyselných systémoch, ktorá má za následok prudký nárast nečistôt v ovzduší.

InfluxDB umožňuje vykonávať agregácie údajov vrátane výpočtu priemerov, súm a iných štatistík^{2,4}, čo uľahčuje analýzu údajov a získavanie kľúčových poznatkov. InfluxDB taktiež umožňuje nastaviť dobu uchovávania dát, čo je užitočné pre udržanie efektivity a optimalizáciu úložiska.

2.2 Definícia úložiska

Dáta o kvalite ovzdušia zo staníc na území mesta Brna. Nejedná sa o verifikované dáta. Stanice sú v správe SHMÚ, mesta Brna a Štátneho zdravotníckeho ústavu Ostrava. Dáta a ich aktualizčný čas sú uvedené nižšie.

- SO₂_1h (Oxid siričitý)
- NO₂_1h (Oxid dusičitý)
- CO_8h (Oxid uhličitý)
- O₃_1h (Ozón)
- PM_{2.5}_1h (polietavý prach o veľkosti pod 2.5 mikrometrov)
- PM₁₀_1h (polietavý prach o veľkosti pod 10 mikrometrov)
- PM₁₀_24h

2.3 Import dát

Nasledujúce kroky popisujú, ako importovať dáta do InfluxDB:

Vytvorenie databáze

Pred importom údajov je potreba vytvoriť databázu, do ktorej budú údaje uložené. To robíme pomocou InfluxDB CLI ale existujú aj iné metódy, ako napr. HTTP API.

```
influx
CREATE DATABASE air_quality
```

Definícia meraní

V InfluxDB sú dáta organizované do meraní. Meranie je základnou jednotkou dát a predstavuje konkrétny typ dát (napr. SO₂, NO₂, CO, atď.). Meranie môže byť vytvorené explicitne pre každý druh dát zvlášť, ale malo by sa automaticky vytvoriť aj pri samostatnom importe.

```
CREATE MEASUREMENT S02
```

Na import dát do InfluxDB používame InfluxDB Line Protocol, ktorý umožňuje importovať dáta vo forme časových radov. Každý záznam by mal obsahovať časovú známku, hodnotu a ďalšie tagy na identifikáciu dát.

V tomto príklade sú importované hodnoty SO₂ z dvoch staníc v časovej známke 1677640800.:

```
S02,station=Brno value=5.2 1677640800
S02,station=Ostrava value=3.8 1677640800
```

2.4 Dotaz nad databázou

Po importe dát vykonávame rôzne otázky a analýzy nad dátami. InfluxDB poskytuje dotazovací jazyk, ktorý umožňuje vykonávať operácie ako filtrovanie a agregácia dát.

```
SELECT MEAN(value) FROM S02 WHERE time > now() - 1d
SELECT MAX(value) FROM CO WHERE time > now() - 3d
SELECT MIN(value) FROM PM10_24h WHERE time > now() - 1w
SELECT SUM(value) FROM NO2 WHERE time > now() - 8h
```

3 Grafová databáze

Dataset: Hlasování zastupitelstva

URL: <https://data.gov.cz/datov%C3%A1-sada?iri=https%3A%2F%2Fdata.gov.cz%2Fzdroj%2Fdatov%C3%A9-sady%2F44992785%2Fadbe37958992dd6ded7545352803e305>

Distribuce datové sady: Je dostupná pouze v JSON

Zvolený produkt: Neo4J verze 5.12.0

Vycházelo se z oficiální dokumentace: <https://neo4j.com/docs/>

3.1 Vysvětlení zvolení databáze

Jako první je potřeba se podívat na strukturu dané datové sady. Kde hlasování zastupitelstva obsahuje potřebné vztahy mezi daty na to aby byli vhodné pro grafovou databázi a tedy neo4j. Specificky se jedná o napojení politických stran na daného politika, dále nepojení politika na dané hlasování a jakým způsobem hlasoval. Tyto vztahy mezi daty nám tedy dovolují je efektivně využít v neo4j.

Systém neo4j nám poté dovoluje vytvářet pro dané politiky jakým způsobem hlasovali na novém hlasování. Tohle je důležitá vlastnost, kvůli které je grafová databáze opět vhodná pro tento dataset. Nemusíme pokaždé ukládat všechny politiky a strany do kterých patří ale můžeme pouze napojovat již existující politiky na nově vytvořené hlasování. Zároveň neo4j umožňuje vytvářet obecný vztah hlasoval a v něm specifikovat jak daný politik hlasoval, tedy ano, ne nebo se zdržel hlasování.

Takovou grafovou databázi poté můžeme využít pro komplexní dotazování na vztahy databázi. Můžeme tedy hledat politiky z některých stran, kteří nějak hlasovali na daném hlasování. Nebo můžeme hledat, kteří politici se neúčastní hlasování. Zároveň se můžeme dotazovat pro dané hlasování kolik každá strana hlasovala ano nebo ne. Neo4j nám taky umožňuje hledat v samotných uzlích a tedy můžeme hledat specifické hlasování podle parametrů jako datum nebo název.

Neo4j nám také umožňuje poté mazat jenom samotné vztahy. Není tedy potřeba mazat třeba celé hlasování, ale stačí pouze odstranit jak politici hlasovali. Nebo pokud politik přejde k jiné straně, stačí pouze změnit jeho vztah na jinou stranu.

Databázi je poté možné škálovat horizontálně. Neo4j nabízí možnost distribuce databáze na více serverech pomocí shardingu. Dále také nabízí vytváření indexů pro atributy a tedy zrychlení dotazování na data podle specifických kritérií.

3.2 Definice úložiště

Neo4j má různé způsoby distribuce od self managed instalaci systému, cloud služby nebo pomocí docker.

Dále je také nabízen v několika verzích, v základní verzi zadarmo není možné vytvářet více databází má tedy pouze systém a základní databázi. Pro práci s databází bude používat základní databázi, kde její jméno je možné změnit v konfiguračním souboru. Pokud bychom pracovali s enterprise verzí neo4j tak bychom vytvořili novou databázi pomocí tohoto cypher příkazu, který vytvoří novou databázi pokud již taková neexistuje.

```
CREATE DATABASE voting IF NOT EXISTS
```

Pro zadávání jednotlivých příkazů pro vytvoření databáze je možné použít buďto GUI webové rozhraní kde je možné zadávat cypher instrukce. Nebo je možné použít cypher-shell pomocí kterého se připojíme na daný server a poté můžeme zadávat cypher instrukce.

Podrobné nastavení databáze je poté možné dělat v neo4j konfiguračním souboru. Pro vložení dat do databáze není potřeba vytvářet jakékoliv schéma databáze, je možné tedy hned začít s importem dat do uzlů a hran pomocí cypher příkazů.

Podrobný popis příkazů pro vytvoření jednotlivých uzlů je možné najít v další sekci import dat. Příklad vytvoření uzlu pro hlasování je následující

```

MERGE (v:Voting {
  code: vote.code,
  number: vote.number,
  datetime: vote.datetime,
  subject: vote.subject,
  result: vote.result
})

```

3.3 Import dat

Tato sekce navazuje na definici úložiště a popisuje přesný import dat. Protože daná datová sada je distribuována jako datový endpoint, který vrací data ve formátu JSON.

APOC knihovna nám umožňuje použít dva způsoby jak daná data importovat. První je stažení dat z datového endpointu a uložení JSON souboru, v jazyce cypher je poté možné daný soubor načíst a z přečtených dat začít vytvářet uzly a hrany. Jako druhou možnost můžeme rovnou načítat data z datového endpointu, není potřeba tedy ukládat JSON soubor. Pro oba způsoby by se pouze změnil začátek příkazu.

Příklad pro napojení na json soubor.

```

CALL apoc.load.json("file:///voting.json") YIELD value
UNWIND value.data AS vote

```

Příklad pro napojení přímo na datový endpoint.

```

WITH "https://kod.brno.cz/zastupitelstvo/" AS url
CALL apoc.load.json(url) YIELD value
UNWIND value.data AS vote

```

Kde vytvoření uzlů a hran vypadá následovně. Pro vložení použijeme příkaz merge, který buďto vloží nový uzel nebo ho aktualizuje. Vytvoříme uzly pro hlasování, politickou stranu a politika. Následně vytvoříme hrany pro to, jak politik hlasoval kde typ hlasování je rozlišen pomocí parametru way, poté vytvoříme hranu pro napojení politika na stranu a napojit jaké členy má politická strana. Nakonec vrátíme výsledné vytvořené uzly. Samotný datový endpoint poté umožňuje přidat parametry pro množství záznamů nebo seřazení od nejnovějších nebo nejstarších. Pro demonstraci na menším počtu dat je tedy import do databáze udělán nad 20 hlasováníma.

```

WITH "https://kod.brno.cz/zastupitelstvo/?limit=20" AS url
CALL apoc.load.json(url) YIELD value
UNWIND value.data AS vote
UNWIND vote.parties AS party
UNWIND party.votes AS voter
MERGE (v:Voting {
  code: vote.code,
  number: vote.number,
  datetime: vote.datetime,
  subject: vote.subject,
  result: vote.result
})
MERGE (p:Party {name: party.name})
MERGE (i:Individual {name: voter.voter})

```

```

MERGE (i)-[:Voted {way: voter.text}]->(v)
MERGE (i)-[:PartOf]->(p)
MERGE (p)-[:Member]->(i)
RETURN i, v, p

```

Tyto dotazy je také možné poslat přímo v dané aplikaci. Neo4j podporuje řadu napojení na různé jazyky. V nich se poté volají právě specifikované dotazy v dané tranzakci. Je tedy možné se připojit na danou databázi a poté na ni posílat dotazy právě v jazyce cypher.

3.4 Dotaz nad databází

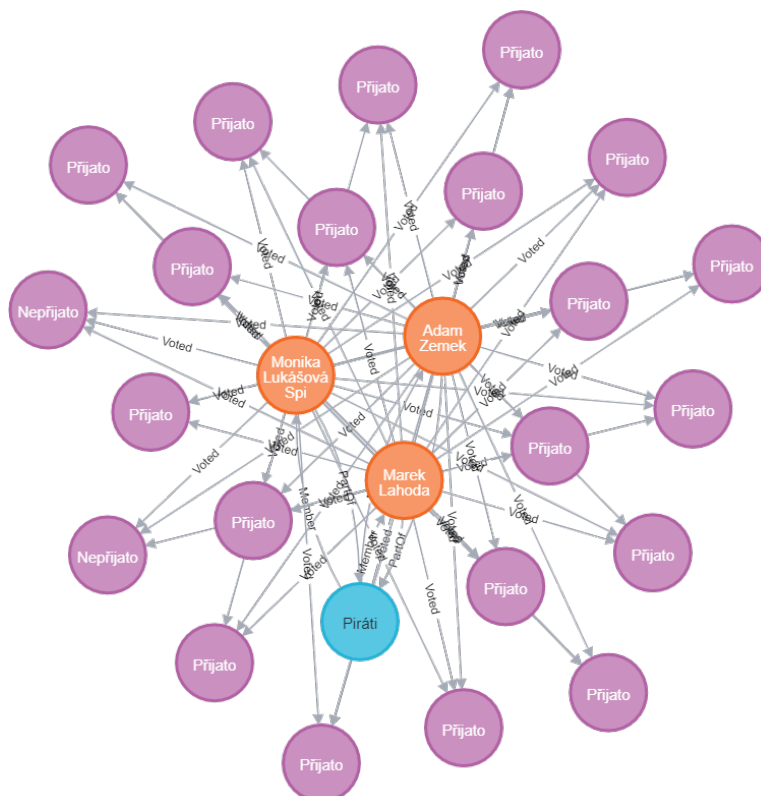
Jako příklad pro dotaz nad databází uděláme ideální příklad proč bylo dobré použít grafovou databázi. Jedná se o příkaz, který nám vrátí všechny politiky kteří jsou součástí strany „Piráti“ kteří hlasovali Ano na některém hlasování.

```

MATCH p=(Party { name: 'Piráti' })-[Member]->()-[r:Voted {way: 'Ano'}]->() RETURN p

```

Tento daný dotaz je poté serverem zodpovězen následovně. Prvně se vyhledá uzel Party podle jména 'Piráti', poté se projdou hrany Member tohoto uzlu pro vyhledání politiků dané strany a následně prochází jejich hrany Voted které jsou typu ano. Kde tyto data jsou serverem shromážděna do výsledného grafu p kde tento graf je tedy výsledek daného dotazu. Kde grafická reprezentace daného grafu je 1. Jedná se pouze o grafickou reprezentaci, při dotazování ve zvoleném jazyce který je podporovaný neo4j drivers je poté výsledek většinou v podobě objektů. Nebo je možné také mít odpověď v podobě JSON.



Obr. 1: Výsledný graf dotazu

4 Dokumentová databáza

Dataset: Nehody s účastí chodců

URL: <https://data.gov.cz/datov%C3%A1-sada?iri=https%3A%2F%2Fdata.gov.cz%2Fzdroj%2Fdatov%C3%A9-sady%2F44992785%2F6ccc7b9817c0125115d7bfe5d7c16617>

Distribúcia dátovej sady: Je dostupná ako CSV, JSON, KML aj Shapefile. Použijeme JSON.

Zvolený produkt: MongoDB

Vychádzali sme z oficiálnej dokumentácie: <https://www.mongodb.com/docs/>

4.1 Vysvetlenie zvolenia databázy

Databázu MongoDB je vhodné použiť medzi inými pri práci s dátovými sadami obsahujúcimi napríklad časové rady, prípadne logy udalostí, či geografické údaje. Nami zvolená dátová sada obsahuje práve určité nezávislé udalosti o nehodách v meste Brno s geografickými údajmi, kde sa udalosť odohrala, kedy nad týmito dátami bude v MongoDB možné vykonávať analýzu udalostí, či vyhľadávanie na základe geografických údajov vďaka vstavanej MongoDB podpore pre tento typ dát.

Nami zvolená dátová sada obsahuje presné časové a geografické údaje, kedy a kde sa odohrala daná nehoda. Taktiež obsahuje aj dodatočné informácie o aktuálnych podmienkach a okolnostiach pri nehode, ako aj počty zranených, či usmrtených ľudí.

MongoDB je dokumentová databáza a dáta v nej sú reprezentované vo formáte BSON (Binary JSON). Preto je výhodné, ak je dátová sada dostupná vo formáte JSON, nakoľko sa jedná o veľmi podobné formáty. Zvolená dátová sada je dostupná vo formáte JSON, preto bude následný import dátovej sady vhodný pre databázu MongoDB.

Dátová sada je aktualizovaná raz ročne a obsahuje všetky dáta od roku 2010. Dáta z dátovej sady by preto bolo vhodné aktualizovať aj u nás raz ročne po tom, ako budú uvoľnené aktualizované dáta. Tým, že dátová sada obsahuje po aktualizácii vždy všetky staré dáta spojené s novo pribudnutými dátami za ďalší rok, bude potrebné zabezpečiť, aby nevznikala žiadna redundancia. Každému riadku dát by bol priradený unikátny identifikátor v rámci databázy, čím by vznikala každoročná duplicita všetkých pôvodných a nových dát. Ďalším riešením tohto problému by mohlo byť aj premazanie dát z predchádzajúceho importu a naplnenie tabuľky aktualizovanými dátami. Tým by sme však mohli prísť o zmenené záznamy, ak by sa tam nejaké nachádzali. Nakoľko sa však jedná o logy udalostí, ktoré sa už odohrali, tak je nepravdepodobné, že by boli významne často upravované ručne.

4.2 Definícia úložiska

Bude potrebné vytvoriť akúsi distribúciu, na ktorú bude možné nasadiť NoSQL databázu MongoDB, napríklad Docker. Po nasadení distribúcie bude potrebné vytvoriť klienta MongoDB databázy. Pripojenie do databázy je možné pomocou Mongo shellu príkazom `mongosh`. Databázu bude následne po pripojení možné vytvoriť pomocou príkazu `use db_nehody`. Keď máme vytvorenú databázu bude možné vytvoriť konkrétnu tabuľku/kolekciu pre našu dátovú sadu.

Kolekciu vytvoríme pomocou príkazu `insertOne`, kedy príkaz pri neexistencii kolekcie danú kolekciu vytvorí:

```
db_nehody.c_nehody.insertOne({
  "OBJECTID": 1,
  "id": 60042102039,
  "d": -601201.157,
  "e": -1164285.355,
  "datum": 1272499200000,
  "den": 4,
  "rok": 2010,
```

```

"mesic": 4,
"zavineni": "chodcem",
"viditelnost": "ve dne, viditelnost nezhoršená vlivem povětrnostních podmínek",
"cas": 1340,
"hodina": 13,
"mesic_t": "duben",
"doba": "den",
"den_v_tydnu": "čtvrtek",
"alkohol": "Ne",
"alkohol_vinik": "ne",
"hlavni_pricina": "nezaviněno řidičem",
"srazka": "srážka s chodcem",
"pricina": "nezaviněná řidičem",
"stav_vozovky": "povrch suchý, neznečistěný",
"povetrnostni_podm": "neztížené",
"rozhled": "dobré",
"misto_nehody": "žádné nebo žádné z uvedených",
"druh_komun": "komunikace sledovaná (ve vybraných městech)",
"usmrceno_os": 0,
"tezce_zran_os": 0,
"lehce_zran_os": 0,
"hmotna_skoda": 32000,
"smrt": 0,
"tz": 0,
"lz": 0,
"smrt_dny": 1,
"smrt_po": null,
"vek_skupina": "19-24",
"chodec_pohlavi": "muž",
"kategorie_chodce": "muž",
"stav_chodce": "nepozornost, roztržitost",
"chovani_chodce": "špatný odhad vzdálenosti a rychlosti vozidla",
"situace_nehody": "přecházení mimo přechod (20 a více metrů od přechodu)",
"prvni_pomoc": "nebylo třeba poskytnout",
"nasledky_chodce": "bez zranění",
"nazev": "Brno-Starý Lískovec",
"POINT_X": 16.571771691,
"POINT_Y": 49.160585186,
"GlobalID": "2a763843-3760-4348-bd1b-2830c4ae7089"
})

```

4.3 Import dát

Ako prvý krok sa musíme pripojiť ku nami špecifikovanej databáze z predchádzajúceho kroku. MongoDB ponúka ovládače pre rôzne jazyky. Napríklad by bolo možné sa pripojiť v jazyku Python pomocou knižnice `pymongo`. Tu by sme si následne špecificky zvolili databázu, ktorú budeme používať, a to by v našom prípade bolo `db_nehody`. Po tom by sme si zvolili aj konkrétnu kolekciu, do ktorej budeme vkladať dáta, čo je v našom prípade `c_nehody`.

Dátová sada je distribuovaná pomocou dátového endpointu, ktorý nám dané dáta vráti vo formáte JSON. Tieto dáta následne spracujeme, aby sme mali kolekciu jednotlivých prvkov. To teda znamená,

že vezmeme dáta z „features“, odkiaľ vyberieme objekt „attributes“.

Pre to, aby sme vkladali dáta bude potrebné, aby sme súčasne aktualizovali alebo vložili nové dáta, bude potrebné vybrať atribút, ktorý budeme používať ako unikátny kľúč. V našom prípade môžeme ako unikátny kľúč zvoliť buď ObjectId, alebo GlobalId.

Pre samotné vloženie dát do databázy použijeme metódu `update`. Prejdeme teda postupne dáta, ktoré sme dostali z dátového endpointu. Pre každý záznam vytvoríme daný príkaz, kde budeme aktualizovať objekt podľa ObjectId a vložíme požadované dáta. Následne ešte pridáme nastavenie `upsert`, ktorý nám zaistí, že pokiaľ daný objekt neexistuje tak ho vytvoríme. To nám zabezpečí to, že vždy budeme môcť aktualizovať databázu s novými dátami, prípadne sa pri ich neexistencii vytvoria.

```
for item in data:
    c_nehody.update_one(
        {"OBJECTID": item["OBJECTID"]},
        {"$set": item},
        upsert=True
    )
```

4.4 Dotaz nad databázou

MongoDB je vhodná na prácu s časovými radami, preto si napíšeme napríklad dotaz nad našou kolekciou, ktorý nám vráti nehody z obdobia 2010-2015 v mestskej časti "Brno-jih", kde niekto prišiel o život. Ďalšou možnosťou by bol aj dotaz nad databázou na základe geologických dát, pre ktoré má MongoDB vstavanú podporu.

```
db_nehody.c_nehody.find({
    "smrt": { $gt: 0 },
    "rok": { $gte: 2010, $lte: 2015 },
    "nazev": "Brno-jih"
})
```

MongoDB získa dáta, ktoré spĺňa kritéria dotazu. V našom prípade vyhľadá dokumenty, ktoré majú "smrt" väčšiu ako 0, rok medzi 2010 a 2015 a "nazev" je "Brno-jih". Pre každý dokument, ktorý zodpovedá tomuto dotazu MongoDB získa dáta zo záznamu vrátane všetkých atribútov.

Výsledky budú v JSON formáte a obsahujú všetky dokumenty, ktoré spĺňajú kritéria dotazu.